Testing y validación

Plan, casos, trazabilidad, defectos y resultados de NovaCycle

15 scripts aprobados	2 defectos corregidos
3 unitarios y 12 de integración	Validación con usuarios pendiente

La verificación automática está aprobada en un Docker de pruebas aislado. La aceptación visual y por perfiles no se declara aprobada hasta que se ejecute y documente.

Plan de pruebas y validación - NovaCycle

ID: PP-NC-2026-01

Versión: 1.0 - 06/11/2026

Estado: en ejecución; la aceptación de usuarios queda pendiente de realizar y firmar.

Responsables: equipo de desarrollo NovaCycle. El docente o un compañero que represente cada perfil participa en la validación.

1. Propósito

Verificar que el sistema cumple los requisitos funcionales de gestión de residuos y comprobar, con usuarios, que los flujos son comprensibles y útiles. La verificación compara resultados observados con resultados esperados; la validación requiere que alguien use el producto y confirme que resuelve su necesidad. Una prueba automática que pasa no reemplaza esa aceptación.

Este plan toma como alcance la tercera entrega de Programación Full Stack y el apartado de testing de Ingeniería de Software del proyecto. Las diapositivas y el video proporcionados por el curso se usan como guía metodológica, no como requisitos funcionales de NovaCycle. El video se pudo identificar y consultar por su descripción pública, pero no se obtuvo una transcripción para atribuirle pasos específicos.

2. Alcance

Se probará

- Registro, aprobación, inicio/cierre de sesión y separación de permisos entre administrador, peón, conductor y operario.
- Altas, consultas, modificaciones y bajas de usuarios, camiones y contenedores, incluidos duplicados y recursos en uso.
- Creación, modificación, baja y asignación de rutas; vinculación de contenedores como paradas y registro de recolección.
- Registro, revisión y cierre de incidencias; listados, alertas y marcadores del mapa.
- Ingresos de residuos, estado de camiones, estado de maquinaria y resúmenes operativos.
- Estadísticas del administrador, acceso público al mapa e instalación básica con Docker.
- Mensajes de error, estado HTTP, persistencia y navegación por rol.

No se afirmará como probado en esta entrega

- Rendimiento bajo carga, estrés, recuperación ante caída del servidor o auditoría de seguridad completa: requieren ambientes, objetivos medibles y herramientas adicionales.
- Exactitud de rutas calculadas por servicios cartográficos externos ni disponibilidad de las teselas de terceros.
- Funcionamiento en todos los navegadores/dispositivos y despliegue en la infraestructura de la institución, hasta ejecutar esos casos allí.
- Satisfacción de usuarios reales, hasta completar las sesiones de aceptación documentadas.

Estos puntos no se consideran aprobados por omisión; quedan como riesgos o pruebas pendientes.

Brechas frente a la consigna detectadas durante el análisis: no se halló CRUD de centros de acopio ni inventario CRUD de maquinaria (solo registro de estados), y el Docker usa MariaDB 11.4 aunque la consigna menciona MySQL 8. Se registran en el informe; no se convierten en casos "aprobados" por falta de implementación o ambiente.

3. Estrategia y justificación

Nivel/técnica	Qué comprueba	Evidencia
Caja blanca, unitaria	Ramas de validación, normalización, permisos de negocio y respuesta del controlador con modelo falso, sin abrir la base.	tests/*_unitarias.php y salida CLI.
Caja negra, integración/API	Entradas y salidas HTTP (JSON y códigos 2xx/4xx), acciones de los cuatro roles y efectos en MariaDB.	tests/*_http.php, consultas de comprobación y salida CLI.
Prueba de interfaz	Formularios, mensajes, redirección, visualización de mapa/alertas y diseño en navegador.	Guion manual y capturas con fecha.
Validación/aceptación	Un representante de cada perfil completa tareas reales sin ayuda del desarrollador y opina si el flujo satisface su necesidad.	Acta de sesión, observaciones y conformidad.
Instalación	La aplicación arranca en una instalación nueva con Docker y se puede acceder a web/base.	Guion manual, comandos y capturas.

La caja blanca se justifica porque las reglas internas (campos obligatorios, rol, normalización y vínculo de contenedor con ruta) pueden probarse con entradas límite y modelos falsos. La caja negra se justifica porque el usuario y el frontend dependen de las respuestas reales de las API, no de cómo estén programadas. Las pruebas de interfaz y aceptación detectan problemas que un script HTTP no ve.

Diseño de casos: clases válidas/inválidas, valores límite (vacío, cero, duplicado), transiciones de estado (pendiente/revisada/cerrada; activo/baja) y matriz de roles. La suite se repite tras cambios como prueba de regresión. No se informa porcentaje de cobertura de código porque no hay medición instrumentada.

4. Ambiente, datos y seguridad de la prueba

- Aplicación: PHP 8.2, JavaScript y CSS sin framework; base MariaDB 11.4; servicios definidos en compose.yaml.
- Las pruebas HTTP usan la URL interna fija http://localhost/Proyecto/proyecto/...; se ejecutan **dentro del contenedor web**.
- Se levanta un proyecto Compose separado (novacycle_qa) con puerto distinto y volumen propio. No se ejecutan pruebas que escriben en la base habitual del equipo.
- Los scripts crean cuentas y recursos temporales con identificadores aleatorios, comprueban datos en SQL y los eliminan al finalizar. Si un script falla antes de limpiar, se inspecciona y descarta **solo** el volumen de QA.
- No se publican contraseñas, cookies, dumps con datos de usuarios ni capturas de sesiones activas como evidencia.

5. Criterios de entrada, salida y suspensión

Entrada: versión identificada en Git; PHP sin errores de sintaxis; servicios web y DB saludables; base QA con esquema/datos de prueba; cuentas de los roles disponibles o creadas por el test.

Aprobación técnica: todos los casos automáticos planificados ejecutados y aprobados; ninguna falla crítica/alta abierta; defectos corregidos repetidos y suite de regresión aprobada. Un caso pasa solo si coinciden código HTTP, ok, datos retornados y cambios persistidos cuando corresponda.

Aprobación funcional: al menos un usuario o representante de cada perfil completa el guion de aceptación, registra observaciones y confirma si el flujo le sirve. Sin esa sesión, el resultado es **pendiente**, no aprobado.

Suspender si no arranca el entorno, si hay riesgo de escribir en la base habitual, si falla la limpieza de datos o si un defecto bloqueante impide los demás casos. **Reanudar** al aislar/restaurar el entorno, registrar el defecto y volver a correr el caso más la regresión afectada.

6. Secuencia y responsabilidades

- Equipo de desarrollo: revisar requisitos y preparar entorno QA.
- Equipo de desarrollo: ejecutar unitarias; corregir fallas.
- Equipo de desarrollo: ejecutar integración/API; registrar resultados y defectos.
- Equipo de desarrollo y representantes de perfiles: probar interfaz y aceptación con los casos manuales.
- Coordinador del equipo: revisar matriz de trazabilidad, pendientes y evidencia; incorporar esta sección al PDF integrador.

La planificación es por etapas y dependencias, sin inventar fechas o firmas de terceros. Si se modifica el sistema después del informe, se actualiza versión, fecha y resultado de las pruebas repetidas.

7. Riesgos y mitigación

Riesgo	Impacto	Mitigación
Pruebas HTTP sobre datos de uso	Alto	Compose y volumen QA independientes; limpieza finally.
Servicios externos del mapa sin internet	Medio	Separar validación de datos/markers de comprobación visual del fondo y trazado.
Tests unitarios confundidos con pruebas de API	Alto	Distinguir explícitamente nivel, entradas y evidencia de cada archivo.
URL HTTP fija en los tests	Medio	Ejecutarlos desde web, no desde PowerShell/XAMPP.
Validación declarada sin usuarios	Alto	Mantener UAT como pendiente hasta obtener acta y observaciones reales.
Cambios posteriores al informe	Medio	Registrar commit y repetir suite y casos manuales afectados.

8. Entregables y fuentes

- casos_y_trazabilidad.md: casos, datos, resultados esperados y matriz requisito-prueba.
- informe_ejecucion.md: ambiente, comandos, resultados obtenidos, defectos y pendientes.
- defectos.md: incidencias reales y plantilla para registrar nuevas.
- output/pdf/Informe_testing_NovaCycle.pdf: copia para anexar al documento integrador de la entrega.

Fuentes consultadas: Proyecto_ESI_2026.pdf (tercera entrega y rúbrica); Pruebas_y_su_documentacion.pdf; Testing_-_VV_del_Software_diap.pdf; Tipos_de_pruebas._niveles_de_prueba_y_plan_de_pruebas.pdf; Verificacion_y_validacion_Testing.pdf; video “¿Cómo hacer un plan de Pruebas? | Tutorial paso a paso” de Full Advanced, https://www.youtube.com/watch?v=apbxGwdXORA (descripción pública, sin transcripción verificada). El plan se adapta a este proyecto estudiantil; cifras y herramientas de los ejemplos didácticos no se trasladan como obligaciones.

Casos de prueba y trazabilidad - NovaCycle

Versión: 1.0 - 06/11/2026. Los identificadores permiten relacionar requisito, script, resultado y defecto. Los datos T... y direcciones example.test son sintéticos.

Casos automatizados

U01 - Registro y login por rol (unitaria, caja blanca)

Precondición: PHP CLI; sin servidor ni base. **Datos:** formulario vacío, email inválido, clave corta, rol administrador no permitido en registro público, CI repetida, cuenta pendiente/rechazada/aprobada y cuatro roles. **Pasos:** ejecutar tests/usuarios_unitarias.php. **Esperado:** entradas inválidas no llegan al modelo; los datos válidos se normalizan y la clave se guarda como hash; pendiente/rechazada/sin rol no ingresan; admin va a admin.html, peón y conductor a cuadrilla.html, operario a operario.html. **Aserción:** el script termina con código 0; cada comprobación lanza error al fallar.

U02 - CRUD y permisos de usuarios (integración, caja negra/gris)

Precondición: web y DB QA; cuentas temporales admin y peón. **Datos:** CI y correo sintéticos. **Pasos:** ejecutar tests/usuarios_http.php; listar con ambos roles; registrar/aprobar; modificar rol; intentar baja propia y baja con incidencia; eliminar cuenta temporal. **Esperado:** solo admin administra, el listado no revela contraseñas, cambios y baja se reflejan en SQL, una sesión con rol anterior pierde acceso, recursos relacionados impiden la baja. **Aserción:** respuesta ok, lectura SQL y salida 0.

U03 - Registro, aprobación y login HTTP real (integración)

Precondición: web y DB QA; admin temporal. **Datos:** cuentas sintéticas de peón, conductor y operario, contraseña correcta e incorrecta. **Pasos:** ejecutar tests/autenticacion_http.php; registrar cada rol por API, intentar login pendiente, aprobar con admin, probar clave incorrecta y luego iniciar sesión con clave válida; consultar una API permitida con la cookie real. **Esperado:** pendiente sin fila en tablas de roles y sin acceso; aprobada con un único rol correcto; contraseña errónea 400; login válido devuelve cookie, rol y destino correctos; la sesión puede consultar su API. **Aserción:** HTTP, JSON, cookie y SQL.

C01 - Camiones: alta, modificación, baja y reactivación (integración)

Precondición: web y DB QA; admin temporal. **Datos:** matrícula aleatoria, capacidad 1000/1500/2000 kg. **Pasos:** ejecutar tests/camiones_db_http.php; crear, modificar, dar de baja y volver a registrar la misma matrícula. **Esperado:** alta 201, cambio y baja 200; la baja no aparece en listado; la matrícula dada de baja reactiva el mismo ID sin duplicar registro; una matrícula activa duplicada se rechaza con 400. **Aserción:** estado HTTP, JSON y SQL.

C02 - Camión asignado: bloqueo y liberación (integración)

Precondición: web y DB QA; admin, conductor y peón temporales. **Datos:** dos camiones, uno asignado a ruta de hoy y otro libre. **Pasos:** ejecutar tests/camion_asignado_http.php; consultar en_uso; intentar editar/dar de baja el asignado, editar/dar de baja el libre, completar asignación y volver a editar el antes asignado. **Esperado:** en uso 409 sin cambios SQL; peón 403; libre editable y dado de baja; ruta completada libera el camión. **Aserción:** HTTP, JSON y SQL.

CT01 - Contenedor vinculado a ruta (integración)

Precondición: web y DB QA; admin temporal. **Datos:** contenedor temporal con coordenadas y ruta temporal. **Pasos:** ejecutar tests/contenedores_ruta_http.php; consultar contenedor libre, crear vínculo de ruta, intentar modificar/dar de baja, quitar vínculo y repetir. **Esperado:** en_uso cambia de 0 a 1; edición/baja mientras está en uso se rechazan con 409; fuera de ruta vuelve a ser editable. **Aserción:** HTTP, JSON y SQL.

CT02 - CRUD real de contenedores por API (integración)

Precondición: web y DB QA; admin y peón temporales. **Datos:** dirección aleatoria, capacidad 800 L, coordenadas válidas/inválidas. **Pasos:** ejecutar tests/contenedores_crud_http.php; intentar altas inválidas, crear, obtener/listar, editar válido/inválido y dar de baja dos veces. **Esperado:** alta 201, consultas/edición/baja válidas; inválidos 400, rol peón 403, recurso dado de baja 404; altas/ediciones/baja comprobadas en SQL. **Aserción:** HTTP, JSON y SQL. No se afirma reactivación por dirección: esa política no existe para contenedores.

R01 - Ruta, asignación y recolección (integración)

Precondición: web y DB QA; admin, peón y conductor temporales. **Datos:** ruta, contenedor, camión y fecha actual generados por el test. **Pasos:** ejecutar tests/rutas_recoleccion_http.php; crear ruta y parada, asignarla, consultar mi_ruta, completar parada, consultar mapa y dar de baja ruta. **Esperado:** la cuadrilla correcta ve su asignación, la parada se completa una vez, la recolección queda en SQL, una parada ajena no se completa y la ruta dada de baja desaparece de listados/mapa. **Aserción:** HTTP, JSON y SQL.

I01 - Incidencias y seguimiento (integración)

Precondición: web y DB QA; ruta asignada y perfiles temporales. **Datos:** desborde en contenedor de ruta, contenedor ajeno, detalle vacío y aviso en otro lugar. **Pasos:** ejecutar tests/incidencias_http.php; registrar, consultar listados/mapa, revisar y cerrar con operario. **Esperado:** dato incompleto/ajeno 400, rol incorrecto 403, registro válido 201; estado pendiente -> revisada -> cerrada en SQL; cerrada sale del mapa y entra al listado de cerradas. **Aserción:** HTTP, JSON y SQL.

S01 - Sesiones simultáneas (integración)

Precondición: web y DB QA; admin y peón temporales. **Pasos:** ejecutar tests/sesiones.php para alternar directamente las sesiones y tests/sesiones_http.php para repetir por API, probar cookies antiguas y cerrar solo una. **Esperado:** cada pestaña mantiene sus permisos; cerrar peón no cierra admin. **Aserción:** rol de sesión, permisos vía API y salida 0.

O01 - Operario y conductor (integración)

Precondición: web y DB QA; perfiles y dos camiones temporales. **Datos:** peso 0 y 175 kg, tipo de residuo, equipo/estado, cambio de estado de camión asignado y ajeno. **Pasos:** ejecutar tests/conductor_operario_http.php. **Esperado:** solo conductor cambia el camión asignado hoy; el camión ajeno queda intacto; solo operario registra ingreso/maquinaria y consulta resumen; ingreso inválido 400 sin guardar; ingreso válido 201 y aumenta ingresos_hoy en 175 kg; los registros quedan vinculados a las cuentas de prueba. **Aserción:** HTTP, JSON y SQL.

R02 - Contrato HTTP de errores de creación (integración)

Precondición: web y DB QA; admin/operario temporales. **Datos:** ruta sin zona, parada sin ruta, asignación sin fecha, ingreso con peso 0. **Pasos:** ejecutar tests/conductor_operario_http.php. **Esperado:** las cuatro acciones devuelven 400 y ok:false, nunca 201, y no insertan registros. **Aserción:** estado HTTP, JSON y conteos SQL.

D01 - Dashboard y alertas del administrador (integración)

Precondición: web y DB QA; admin temporal. **Datos:** recursos e incidencia temporales. **Pasos:** ejecutar tests/dashboard_alertas_http.php; consultar dashboard antes/después de crear datos, comprobar incrementos y acceso al mapa público. **Esperado:** rutas y recolecciones del día +1, ingresos +500 kg, contenedores por estado +1, camión disponible +1, incidencias pendientes +1 y luego -1 al cerrar; la incidencia abierta aparece en mapa y la cerrada no; rol no admin recibe 403. **Aserción:** HTTP y JSON comparados con datos insertados en SQL. La creación visual de tarjetas de alerta en el DOM requiere M02.

B01 - Reglas de operaciones y CRUD (unitarias, caja blanca)

Precondición: PHP CLI; sin servidor ni base. **Datos:** nombres vacíos/con espacios, IDs cero, contenedor ajeno, incidencia en ruta y en otro lugar. **Pasos:** ejecutar tests/operaciones_unitarias.php, tests/usuarios_unitarias.php y tests/crud_unitarias.php. **Esperado:** las entradas inválidas no pasan al modelo; las válidas se normalizan; altas, modificaciones y bajas de camiones/contenedores respetan el bloqueo de recursos en uso. **Aserción:** modelos falsos y salida 0.

Casos manuales de interfaz y validación

Cada caso manual debe registrar **fecha, persona/perfil, navegador, versión/commit, resultado real, captura y estado** (aprobado, falló, bloqueado o pendiente). No se considera aprobado solo por estar escrito.

Orden sugerido: M02 crea y asigna una ruta con dos contenedores para hoy; M03 reporta la incidencia; volver a M02 para observar la alerta; M01 comprueba el mapa; M04 cierra la incidencia; volver a M01/M02 para comprobar que desapareció de abiertos y bajó el contador. M05 se hace en la máquina destino.

M01 - Landing y mapa público

Precondición: M02 dejó asignada para hoy una ruta con dos contenedores geolocalizados; M03 creó una incidencia abierta para uno de ellos. **Pasos:** abrir la landing sin sesión; identificar ruta, contenedores e incidencia; acercar el mapa y revisar el trayecto entre ambas paradas. Repetir tras M04. **Esperado:** no exige login; marcadores corresponden a registros activos; la incidencia cerrada deja de verse; la traza sigue calles transitables y no cruza agua; si falla el servicio externo, se informa sin romper la página. **Validación:** preguntar al usuario si entiende qué muestran los símbolos y si puede ubicar su zona.

M02 - Flujo del administrador

Pasos: entrar como admin; aprobar usuario; crear camión, dos contenedores y ruta; vincular ambos contenedores como paradas y asignar cuadrilla para hoy; intentar editar/borrar camión y contenedor en uso; consultar dashboard. Tras M03, comprobar alerta activa; tras M04, comprobar el cambio de contador. **Esperado:** acciones y mensajes coherentes, bloqueo visible para recursos en uso, datos actualizados al volver a abrir cada sección. **Validación:** el usuario puede encontrar las secciones y completar una asignación sin ayuda.

M03 - Flujo de cuadrilla

Pasos: entrar como peón/conductor; ver ruta de hoy; completar una parada; reportar incidencia en su contenedor y otra ubicación; intentar acción exclusiva de admin/operario. **Esperado:** tareas y contador coinciden con la ruta asignada; mensaje de incidencia enviada; permisos negados donde corresponde. **Validación:** la cuadrilla sabe qué parada le toca y cómo reportar un problema.

M04 - Flujo del operario

Pasos: entrar como operario; consultar incidencias; revisar/cerrar una; registrar peso de residuos y estado de maquinaria; comprobar resumen. **Esperado:** datos actualizados, estados claros, entradas inválidas con mensaje útil. **Validación:** el operario puede distinguir pendiente de revisada/cerrada y entender el resumen.

M05 - Instalación y compatibilidad mínima

Pasos: en equipo distinto o entorno nuevo, configurar .env, iniciar docker compose up --build -d, abrir landing, login y phpMyAdmin; repetir M01-M04 en al menos dos tamaños de pantalla y dos navegadores. **Esperado:** servicios saludables, sin datos/contraseñas expuestos, interfaz legible y acciones funcionales. **Estado:** pendiente hasta ejecutarse realmente en ese equipo.

Matriz de trazabilidad

Los requisitos provienen de Proyecto_ESI_2026.pdf, secciones 9.1 y 9.4 de la tercera entrega y de las funciones implementadas. Auto no significa que la validación con usuarios esté hecha.

ID requisito	Requisito	Casos relacionados
RF01	Registro, sesión, roles y CRUD de usuarios	U01, U02, U03, S01, M02, M03, M04
RF02	CRUD de camiones y protección de camión asignado	C01, C02, B01, M02
RF03	CRUD de contenedores y vínculo con ruta	CT01, CT02, B01, M02
RF04	API de recolección, rutas y paradas	R01, R02, B01, M02, M03
RF05	Incidencias: registro, seguimiento y listados	I01, D01, M03, M04
RF06	Mapa interactivo con contenedores e incidencias	I01, R01, D01, M01
RF07	Informes y panel de estadísticas	D01, O01, M02, M04
RF08	Sistema de alertas	D01, M02
RF09	Operario: ingresos y maquinaria; conductor: estado de camión	O01, M03, M04

ID requisito	Requisito	Casos relacionados
RF10	CRUD de centros de acopio y maquinaria básica (segunda entrega)	Sin implementación completa; brecha registrada en el informe
RNF01	Seguridad básica: permisos y sesiones	U01, U02, U03, C02, CT02, I01, S01, O01
RNF02	Instalación con Docker	M05
RV01	Utilidad/usabilidad para perfiles reales	M01, M02, M03, M04, M05

Brechas a revisar: la presentación del bloqueo de camión asignado y la visualización/claridad de alertas requieren observación manual además de API; el trazado vial externo no está cubierto por los tests HTTP; la conformidad de usuarios sigue pendiente hasta que se ejecute el guion.

Registro de defectos - NovaCycle

Versión de revisión: 06/11/2026. Se registran solo problemas observados en código y comprobados con pruebas; no se agregan defectos hipotéticos como si hubieran ocurrido.

DEF-001 - Una operación rechazada respondía como exitosa

- **Severidad:** media (contrato HTTP incorrecto para el frontend y clientes de la API).
- **Área:** Api_Operaciones/api_operaciones.php.
- **Precondición:** usuario con rol permitido.
- **Reproducción:** enviar POST ingreso con peso_kg=0, o POST crear_ruta sin zona, crear_parada sin IDs, asignar_ruta sin fecha.
- **Esperado:** JSON ok:false, HTTP 400, ningún registro nuevo.
- **Observado antes de corregir:** por revisión del código, esas ramas pasaban 201 fijo a responder(...) aunque el controlador devolvía ok:false. No se conserva una captura de ejecución anterior al cambio; esta observación se basa en el código previo.
- **Causa:** código de éxito fijado en la capa HTTP, sin consultar el resultado del controlador. Otras acciones fallidas también respondían 200.
- **Corrección:** todas las acciones mutantes de operaciones usan 400 cuando ok es falso; las creaciones válidas conservan 201 y las modificaciones válidas 200.
- **Reprueba:** R02 en tests/conductor_operario_http.php comprueba los cuatro casos y ausencia de inserciones; R01 comprueba errores de recolección. Resultado: aprobado en Docker QA aislado.
- **Estado:** corregido y repruebado.

DEF-002 - Conductor podía cambiar un camión ajeno

- **Severidad:** alta (permiso por rol insuficiente para proteger recursos de otra cuadrilla).
- **Área:** estado_camion y ModeloOperaciones::actualizarCamion.
- **Precondición:** sesión de conductor y un camión activo que no esté asignado a ese conductor hoy.
- **Reproducción:** enviar POST estado_camion con el ID del camión ajeno y un estado válido.
- **Esperado:** ok:false, HTTP 400; el estado del camión ajeno permanece intacto.
- **Observado antes de corregir:** la API comprobaba el rol conductor, pero el UPDATE filtraba solamente por ID y activo=1; cualquier conductor podía afectar otro camión. Observación por revisión del código, sin afirmar una ejecución previa del ataque.
- **Causa:** faltaba asociar la actualización a la asignación del día y a la CI autenticada.
- **Corrección:** el UPDATE exige una asignación de hoy cuyo ci_conductor coincide con la sesión; el controlador pasa la CI y la API devuelve 400 si se rechaza.
- **Reprueba y estado:** O01 en tests/conductor_operario_http.php usa dos camiones: el asignado cambia, el ajeno no; comprueba respuesta y SQL. Aprobado en Docker QA aislado; corregido y repruebado.

Pendientes de observación

No se detectaron fallas en los 15 scripts ejecutados después de las correcciones. Esto **no prueba ausencia de defectos**. Todavía falta ejecutar los casos de interfaz y aceptación M01-M05, especialmente trazado vial real, tarjetas visuales de alerta, legibilidad móvil y experiencia de cada perfil.

Plantilla para nuevos defectos

- **ID y título:**
- **Fecha, responsable y versión/commit:**
- **Severidad:** crítica / alta / media / baja.
- **Módulo y ambiente:**
- **Precondiciones y datos de prueba:**
- **Pasos exactos para reproducir:**
- **Resultado esperado:**
- **Resultado observado:**
- **Evidencia:** captura, respuesta HTTP o registro de consola sin secretos.
- **Estado:** abierto / en corrección / corregido / repruebado / cerrado.
- **Caso de regresión asociado:**

Informe de ejecución y validación - NovaCycle

ID: IE-NC-2026-01

Fecha: 06/11/2026

Versión examinada: base Git 42a65e7 de segundo_avance, con los cambios de testing y corrección descritos en este informe.

Estado general: verificación automatizada aprobada; validación con usuarios pendiente.

1. Ambiente y método

Se construyó un proyecto Docker independiente llamado novacycle_qa_extra_20261106, con web en puerto 18080 y volumen MariaDB propio. Se ejecutaron las pruebas de forma secuencial dentro del contenedor web, PHP 8.2.35 y MariaDB 11.4 (imagen declarada en compose.yaml). La base se inicializó con database/datos_demo.sql. El proyecto Docker habitual proyecto no se usó para estas pruebas. Los tests HTTP insertan datos temporales, verifican las respuestas y, cuando corresponde, SQL; usan bloques finally para limpiar.

Cada script aprobado terminó con código de salida 0. Un script de integración debe ejecutarse solo con NOVACYCLE_TEST_ISOLATED=1 y una base separada. No se midió cobertura porcentual de líneas/ramas ni rendimiento; no se informa un porcentaje ficticio.

2. Resultado de las pruebas automáticas

Script	Nivel	Resultado observado
operaciones_unitarias.php	Unitaria	Aprobó: validaciones de rutas, paradas e incidencias.
usuarios_unitarias.php	Unitaria	Aprobó: registro, login por cuatro roles y validaciones CRUD.
crud_unitarias.php	Unitaria	Aprobó: alta, edición, baja y bloqueo de camiones/contenedores en uso.
sesiones.php	Integración/componente	Aprobó: alternancia directa de dos sesiones asociadas a cuentas QA.
sesiones_http.php	Integración/API	Aprobó: dos sesiones independientes y permisos.
usuarios_http.php	Integración/API	Aprobó: permisos, CRUD y persistencia de usuarios.
autenticacion_http.php	Integración/API	Aprobó: registro real, aprobación, login y cookie para peón/conductor/operario.
contenedores_ruta_http.php	Integración/API	Aprobó: bloqueo y liberación de contenedores vinculados.
contenedores_crud_http.php	Integración/API	Aprobó: CRUD real, validaciones, permisos y persistencia de contenedores.
camiones_db_http.php	Integración/API	Aprobó: edición, baja, reactivación y matrícula duplicada.
camion_asignado_http.php	Integración/API	Aprobó: bloqueo de camión en uso, liberación y persistencia.
rutas_recoleccion_http.php	Integración/API	Aprobó: ruta, parada, asignación, recolección, permisos y SQL.
incidencias_http.php	Integración/API	Aprobó: registro, seguimiento, mapa y cierre.
conductor_operario_http.php	Integración/API	Aprobó: permisos, camión propio/ajeno, ingreso, maquinaria, resumen y errores 400.
dashboard_alertas_http.php	Integración/API	Aprobó: deltas de métricas, datos de origen de alertas y mapa público.

Resumen verificable: 15 scripts ejecutados, 15 aprobados, 0 fallidos (3 unitarios y 12 de integración/componente/API). Cada script contiene varias aserciones; 15 no es el número de aserciones ni un porcentaje de cobertura. Los datos del dashboard se comprobaron mediante incrementos respecto del estado inicial, no con números fijos que dependan del respaldo.

3. Defectos encontrados y regresión

Se detectaron y corrigieron dos defectos antes del cierre de la ejecución: DEF-001 (respuestas HTTP 201/200 en operaciones rechazadas) y DEF-002 (conductor podía modificar un camión no asignado). Los casos negativos nuevos y el resto de la suite pasaron después de las correcciones. Detalle y reproducción en defectos.md.

No se declara que todo el sistema está libre de errores: los tests automáticos no observan el DOM, el trazado visual del mapa ni la facilidad de uso.

4. Validación e interfaz: estado honesto

Caso	Qué debe demostrar	Estado actual
M01	Landing y mapa comprensibles; traza vial razonable.	Pendiente de sesión visual.
M02	Admin completa aprobación, asignación y consulta de alertas sin ayuda.	Pendiente de sesión con usuario.
M03	Peón/conductor entienden la ruta, paradas e incidencia.	Pendiente de sesión con usuario.
M04	Operario distingue estados y comprende ingreso/resumen.	Pendiente de sesión con usuario.
M05	Instalación en otra máquina, dos tamaños y dos navegadores.	Pendiente de ejecución en equipo destino.

La respuesta JSON del mapa y los contadores que alimentan alertas sí fueron comprobados. La presentación del mapa y las tarjetas de alerta, la compatibilidad y la satisfacción de usuarios no se marcarán como aprobadas hasta observarlas.

Acta breve para cada sesión de aceptación

- Registrar fecha, persona (o perfil simulado), navegador/dispositivo y versión.
- Entregar el objetivo de M01-M04 sin explicar dónde hacer clic.
- Anotar pasos que pudo completar, tiempo aproximado, dudas, errores y frases textuales breves del usuario.
- Guardar capturas sin contraseñas ni datos personales y relacionarlas con el ID de caso.
- Preguntar: “¿Pudiste completar la tarea?” y “¿Qué cambiarías?”. Registrar respuesta y decisión del equipo.
- Marcar aprobado/falló/bloqueado; si hubo falla, abrir defecto y repetir tras corregir.

5. Reproducción en PowerShell

Desde C:\xampp\htdocs\Proyecto\proyecto, con Docker Desktop abierto y .env configurado:

```
$env:APP_PORT = '18080'
docker compose -p novacycle_qa_local up -d --build web db
docker compose -p novacycle_qa_local ps
$pruebas = @(
  'operaciones_unitarias.php', 'usuarios_unitarias.php', 'crud_unitarias.php',
  'sesiones.php',
  'sesiones_http.php', 'usuarios_http.php', 'autenticacion_http.php',
  'contenedores_ruta_http.php', 'contenedores_crud_http.php',
  'camiones_db_http.php', 'camion_asignado_http.php',
  'rutas_recoleccion_http.php', 'incidencias_http.php',
  'conductor_operario_http.php', 'dashboard_alertas_http.php'
)
foreach ($prueba in $pruebas) {
  docker compose -p novacycle_qa_local exec -T -u www-data `
    -e NOVACYCLE_TEST_ISOLATED=1 web php "tests/$prueba"
  if ($LASTEXITCODE -ne 0) { throw "Falló $prueba" }
}
```

El nombre novacycle_qa_local crea un volumen distinto del proyecto habitual. **No uses -p proyecto ni ejecutes estos tests HTTP sobre tu base de uso.** Si todo salió bien y no necesitás investigar datos de prueba, eliminá solo la copia QA:

```
docker compose -p novacycle_qa_local down -v
Remove-Item Env:APP_PORT
```

down -v se reserva para esa copia QA: sobre el proyecto habitual borraría sus datos. Para la defensa, conservá este informe y un registro de salida/captura de la ejecución más las actas M01-M05 que realmente completes.

6. Brechas de requisitos detectadas

- El proyecto pide CRUD de centros de acopio y maquinaria básica en la segunda entrega. En este repositorio el centro solo se menciona en textos del frontend; no se encontró tabla, API ni formulario CRUD. Para maquinaria existe registro de estado, pero no un CRUD de inventario. No hay caso automatizado que pueda aprobar esas funciones ausentes. Confirmar con el docente si siguen siendo obligatorias o implementarlas antes de afirmar cumplimiento total.
- La consigna especifica MySQL 8, mientras compose.yaml usa MariaDB 11.4. Las pruebas demuestran funcionamiento en MariaDB; **no** demuestran compatibilidad con el servidor MySQL 8 de la institución. M05 debe ejecutarse en ese ambiente o acordar la sustitución con el docente.

Estos puntos son brechas de alcance/ambiente, no fallos de los 15 scripts ni defectos runtime inventados.

7. Decisión de cierre

Verificación automatizada: aprobada en el entorno descrito.

Validación por usuarios: pendiente, sin firma ni evidencia observacional todavía. **Aprobación final del producto:** no declarada; requiere completar M01-M05, resolver/aclarar las brechas de requisitos y repetir la regresión si se modifica código.